ROXO TRANSLATIONS

CERTIFICATION OF TRANSLATION ACCURACY

Document Type: Article
Source Language: Portuguese
Target Language: English
Number of pages: 01

ata American Translators Association 273295 The Voice of Interpreters and Translators

Roxo Translations ("we" or "our") is Corporate Member 273295 of the American Translators Association (ATA), and is a professional translation agency. We are not related in any way to the client for whom this translation was completed. We hereby certify that the document described above has been translated by our experienced and qualified professional contract translator, fluent in and competent to translate from the source to the target languages. In our best judgment, the translated text accurately reflects the content, meaning and style of the original text, and constitutes in every respect a correct, true and complete translation of the original document.

Our certification relates to the correctness of the translation only. We do not guarantee that the original document is genuine, nor do we guarantee that the statements contained in the original document are true. Furthermore, we assume no liability for the way in which the translation is used by our client or any third party, including end-users of the translation.

A copy of the translation is attached to this certificatition.

Sincerely,

PROFESSIONAL TRANSLATION SERVICES

Daniela Lopes
Roxo Translations
Dated: April 14, 2022
State of Florida

County of Orange

A notary public or other officer completing this certificate verifies only the identity of the individual who signed the document to which this certificate is attached, and not the truthfulness, accuracy, or validity of that document.

Subscribed and sworn to (or affirmed) before me on April 14, 2022, by Daniela Lopes proved to me on the basis of satisfactory evidence to be the person who appeared before me.

NOTARY PUBLIC
Adriana Ferreira de Matos Rates
Comm# HH011788
Expires: 6/17/2024
STATE OF FLORIDA

Notary Public Signature

Technology Leadership Council Brazil
IBM Academy of Technology Affiliated
Mini Paper Series Year 14
June, 2019 - No. 339

Container Networks

Ricardo Nunes Ferreira

Today, when we think about the work of a network analyst, we usually associate their routine with the implementation and operation of switches, routers, firewalls, load balancers, among others. This knowledge is and will still be essential for LAN and WAN networks, but for those who also want to work in projects related to data center and cloud, it will also be necessary to incorporate in their skills the knowledge of container networks. Containers are based on a method of operating system virtualization that allows you to run a application and its dependencies on isolated resource processes. There are several advantages such as reliability, consistency of deployment across different environments, lower consumption of computing resources, and adherence to microservices architecture to build more resilient and scalable cloud applications.

These factors explain the large adoption by application development and operation (DevOps) teams.
Currently, two networking standards for containers stand out:

1) Container Network Model (CNM) proposed by Docker, one of the leading container technology companies;
2) Container Network Interface (CNI) proposed by CoreOS, the company responsible for the "rkt" runtime and adopted by several container orchestration and management projects such as Kubernetes, Cloud Foundry and Apache Mesos.

In this mini paper we will provide an introduction only of the CNM model, which has three entities:

1) Sandbox: includes the container's network implementation (Linux network namespace);
2) Endpoint: the container interface in a given network (Network);
3) Network: a collection of endpoints that can communicate with each other.

Network and IP Address Management (IPAM) drivers developed by Docker or third parties make it possible to implement the CNM model in an environment of containers. While the network drivers create and remove networks, the IPAM drivers are responsible for IP addressing them and providing IP addresses to the containers.

There are two types of drivers: Built-In and Plug-In.

1) Built-In: developed by Docker and available natively through five drivers:
   a) Bridge: Through a Linux bridge (virtual implementation of a switch within the Linux kernel) it enables containers to communicate internally on a single host and communicate externally through port publishing with the iptables tool, native in Linux.
   b) Overlay: uses the VXLAN protocol to encapsulate the layer 2 frames in UDP packets and enables container communication between different hosts. The GOSSIP protocol is used as the control plane.
   c) Host: The containers share with the host the same Linux network namespace (IP addressing, routing tables, etc.).
   d) MACVLAN: associates the containers directly with the host's external network. Each container receives addressing from this external network and uses a gateway outside the host that makes communication between different networks possible.
   e) None: when the container has no connection to the network.
2) Plug-Ins: developed by companies or the Kubernetes community as an alternative to the Overlay driver. As an example, there is the Calico Plug-In, which uses IP-in-IP and BGP protocols for communication between hosts and enables the implementation of security directives for traffic control. It is used in both public cloud and private cloud container orchestration solutions.

It is recommended to start the study of these various networking technologies by first understanding the concept of containers and later moving on to topics such as Kubernetes, Calico, Istio etc. This knowledge will be useful in learning about various cloud products and services. These steps will become easier as the basic concepts are assimilated.

To learn more

https://success.docker.com/article/networking
https://docs.projectcalico.org

Ricardo Nunes Ferreira is an IT architect at IBM Brazil with over 20 years of experience in Information Technology (IT).
He holds an undergraduate degree in Computer Science by Unicsul with several certifications in IT.
Graduate degree in Strategic Business Management by FIAP.
He currently leads complex outsourcing solutions at IBM.
For more information, send an email to **ricardon@br.ibm.com**

Technology Leadership Council Brazil
IBM Academy of Technology Affiliated
Mini Paper Series Ano 14
Junho, 2019 _ Nº 339

Redes para containers

Ricardo Nunes Ferreira

Hoje, quando pensamos na atuação de um analista de redes, geralmente associamos o seu dia a dia com a implementação e operação de switches, roteadores, firewalls, balanceadores de carga, dentre outros. Este conhecimento é e ainda será essencial para redes LAN e WAN, mas para aqueles que também desejam atuar em projetos relacionados a *datacenter* e nuvem também será necessário incorporar em seus *skills* o conhecimento de redes para *containers*. Os *containers* são baseados em um método de virtualização do sistema operacional que permite executar um aplicativo e suas dependências em processos com recursos isolados. Há diversas vantagens como confiabilidade, consistência de implantação em diferentes ambientes, menor consumo de recursos computacionais e aderência à arquitetura de *microservices* para construir aplicações em nuvem mais resilientes e escaláveis. Estes fatores explicam a grande adoção pelas equipes de desenvolvimento e operação de aplicações (DevOps).

Atualmente se destacam dois padrões de rede para *containers*:

1) Container Network Model (CNM) proposto pela Docker, uma das principais empresas de tecnologia para *containers*;
2) Container Network Interface (CNI) proposto pela CoreOS, empresa responsável pelo *runtime* "rkt" e adotado por vários projetos de orquestração e gestão de *containers* como Kubernetes, Cloud Foundry e Apache Mesos.

Neste *mini paper* forneceremos uma introdução somente do modelo CNM, que possui três entidades:

1) Sandbox: contém a implementação de rede do *container* (Linux network namespace);
2) Endpoint: a interface do *container* em uma determinada rede (Network);
3) Network: uma coleção de *endpoints* que podem comunicar entre si.

Os *drivers* de rede e de IP Address Management (IPAM) desenvolvidos pela Docker ou terceiros possibilitam a implementação do modelo CNM em um ambiente de *containers*. Enquanto os *drivers* de rede criam e removem as redes, os drivers de IPAM são responsáveis pelo endereçamento IP das mesmas e fornecimento de endereços IP aos *containers*.

Existem dois tipos de *drivers*: Built-In e Plug-In.

1) Built-In: desenvolvidos pela Docker e disponíveis nativamente através de cinco *drivers*:
   a) Bridge: Através de uma Linux *bridge* (implementação virtual de um switch dentro do kernel do Linux) possibilita que *containers* se comuniquem internamente em um único *host* e se comuniquem externamente através da publicação de portas com a ferramenta *iptables*, nativa no Linux.
   b) Overlay: utiliza o protocolo VXLAN para encapsular os frames Layer 2 em pacotes UDP e possibilita a comunicação de *containers* entre diferentes *hosts*. O protocolo GOSSIP é usado como plano de controle.
   c) Host: os *containers* compartilham com o *host* o mesmo Linux *network namespace* (endereçamento IP, tabelas de roteamento etc.).
   d) MACVLAN: associa os *containers* diretamente com a rede externa do *host*. Cada *container* recebe endereçamento desta rede externa e utiliza um *gateway* fora do *host* que possibilita a comunicação entres diferentes redes.
   e) None: quando o *container* não tem conexão com a rede.
2) Plug-In: desenvolvidos por empresas ou pela comunidade Kubernetes como alternativa ao *driver* Overlay. Como exemplo, há o Plug-In Calico que utiliza os protocolos IP-in-IP e BGP para a comunicação entre *hosts* e possibilita a implementação de diretivas de segurança para controle de tráfego. É utilizado nas soluções para orquestração de *containers* em nuvem pública e em nuvem privada.

Recomenda-se iniciar o estudo destas diversas tecnologias de rede entendendo primeiramente o conceito de *containers* e posteriormente avançando em temas como Kubernetes, Calico, Istio etc. Esse conhecimento será útil no aprendizado de diversos produtos e serviços de nuvem. Estes passos se tornarão mais fáceis à medida que os conceitos básicos forem sendo assimilados.

Para saber mais

https://success.docker.com/article/networking
https://docs.projectcalico.org

Ricardo Nunes Ferreira é arquiteto de TI na IBM Brasil com mais de 20 anos de experiência em Tecnologia da Informação (TI).
É formado em Ciências da Computação pela Unicsul com diversas certificações em TI.
Pós-graduado em Gestão Estratégica de Negócios pela FIAP.
Atualmente, lidera soluções complexas de outsourcing na IBM.
Para maiores informações, envie e-mail para **ricardon@br.ibm.com**